Document Loader In LangChain

Prepare data for meaningful retrieval

CONTENT

Document Loaders in LangChain

Document Loaders are the components in **Langchain** that are used to load data from various sources into a standardized format (usually as **Document objects**), which can then be used for chunking, embedding, retrieval and generation.

This **Document Object** has two parts:

- ***page_content***, which holds the text
- ***metadata***, which includes useful details like where the data came from.

Depending upon the types of data sources, we have different **classes** to load documents. For example: pdf, word, CSV files, web pages, etc. Some of the most common document loaders are:

- Text Loaders
- PyPDF Loader
- WebBase Loader
- CSV Loader
- ***Directory Loader***

1. Working with TextLoader

TextLoader is a simple and commonly used document loader in LangChain that reads plain text (**.txt**) files and converts them into LangChain **Document Objects**. It is ideal for loading chat logs, scraped text, code snippets, or any plain text data into LangChain pipeline.

```
!uv add langchain-community -q
```

```
# importing the TextLoader library from langchain
from langchain_community.document_loaders import TextLoader

loader = TextLoader(file_path="../1_Datasets/Harry_Potter.txt",
encoding="utf-8")
docs = loader.load()
```

```
print("The data type of the document:", type(docs))
print("The length of documents (number of total documents):", len(docs))
print(docs)
```

The docs (**Document Object**) contains two parts: ***metadata*** & ***page_content***. *Let's extract them:*

```
print("Metadata of documents: \n", docs[0].metadata)
print("\nText content: \n", docs[0].page_content[:200])
```

2. Working with PyPDFLoader

PyPDFLoader is another document loader in LangChain that loads content from PDF and converts each page into LangChain **Document Objects**. It uses the **PyPDF** library under the hood. So we need to **install** PyPDF library using the following command:

```
!uv add pypdf
```

Now, let's implement the **PyPDFLoader** for loading **.pdf** files:

```
# importing the PyPDFLoader library from langchain
from langchain_community.document_loaders import PyPDFLoader

pdf_loader =
PyPDFLoader(file_path="../1_Datasets/Hands_On_Large_Language_Models.pdf"
)
pdf_docs = pdf_loader.load()
```

```
print("The data type of the document:", type(pdf_docs))
print("The length of documents (number of total documents):",
len(pdf_docs))
print(pdf_docs)
```

```
The data type of the document: <class 'list'>
The length of documents (number of total documents): 428
[Document(metadata={'producer': 'Antenna House PDF Output Library...
```

The **length of the document** here is equal to the **number of pages** present in the pdf file.

The docs (**Document Object**) contains two parts: ***metadata*** & ***page_content***. Let's extract the **metadata** and **page content** of the very first document:

```
print("Metadata of documents: \n", pdf_docs[0].metadata)
print("\nText content: \n", pdf_docs[0].page_content[:200])
```

Based on the use case, we can use following pdf loaders:

Use Case	Recommended Loader
Simple, clean PDFs	**PyPDFLoader**
PDFs with tables/columns	**PDFPlumberLoader**
Scanned/Image PDFs	**UnstructuredPDFLoader** or **AmazonTextractPDFLoader**
Need layout and image data	**PyMuPDFLoader**

3. Working with WebBaseLoader

WebBaseLoader is a document loader in LangChain that is used to load and extract text content from web pages (URLs). It uses **BeautifulSoup** under the hood to parse the HTML and extract the text. So we first need to install the **bs4** library:

```
!uv add bs4
```

Now, let's implement the **WebBaseLoader**:

```
# importing the WebBaseLoader library from langchain
from langchain_community.document_loaders import WebBaseLoader

web_loader =
WebBaseLoader(web_path="https://en.wikipedia.org/wiki/Harry_Potter")
Web_docs = web_loader.load()
```

```
print("The data type of the document:", type(Web_docs))
print("The length of documents (number of total documents):",
len(Web_docs))
print(Web_docs)
```

Let's make a LLM call and ask few questions:

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()
```

```
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
OPENROUTER_BASE_URL = "https://openrouter.ai/api/v1"

openai = ChatOpenAI(
    model = "openrouter/owl-alpha", # using the free model
    api_key=OPENROUTER_API_KEY,
    base_url=OPENROUTER_BASE_URL
)
```

```
# Making the prompt ready
prompt = """
Answer the following questions:
{questions}
based on the provided context:
{context}
"""
```

```
# Asking LLM to answer the question
response = openai.invoke(prompt.format(
    questions = "When was Harry Potter written?",
    context = Web_docs[0].page_content)).content
```

```
# Lets display the response
from IPython.display import Markdown
display(Markdown(response))
```

```
Harry Potter was written by J. K. Rowling during the 1990s and early
2000s. In particular, each book of the series was completed in that
period: the first book, Harry Potter and the Philosopher's Stone, was
finished in 1995 and published in 1997, and the final book,
Harry Potter and the Deathly Hallows, was finished and published in
2007.
```

4. Working with CSVLoader

CSVLoader is a document loader in LangChain used to load CSV files into LangChain Document objects, one per row, by default. Here the implementation of **CSVLoader** to load .csv files:

```
# importing the CSVLoader library from langchain
from langchain_community.document_loaders import CSVLoader

csv_loader = CSVLoader(file_path="../1_Datasets/top_10_countries.csv",
encoding="utf-8")
csv_docs = csv_loader.load()
```

```
print("The data type of the document:", type(csv_docs))
print("The length of documents (number of total documents):",
len(csv_docs))
```

```
The data type of the document: <class 'list'>
The length of documents (number of total documents): 10
```

The length of the document here is equal to the number of samples (rows) present in the csv file.

```
# Extracting the page_content from each row (Document)
for doc in csv_docs:
    print(doc.page_content)
    print() # For extra line
```

5. Working with Directory Loader

DirectoryLoader is another document loader that allows us to load multiple documents from a directory (folder) of files. It takes **path** and **glob** as parameters. The **glob** parameter is used to specify a file-matching pattern so that the loader knows which files to load from the directory.

Glob Pattern	What it Loads
***.txt**	All **.txt** files in the root directory
***.pdf**	All **.pdf** files in the root directory
data/*.csv	All **.csv** files in the **data/** directory
****/*.txt**	All **.txt** files in all subfolder
****/***	All files (any type, all folder)

Here is the implementation of **DirectoryLoader**:

```
from langchain_community.document_loaders import DirectoryLoader,
PyPDFLoader

dir_loader = DirectoryLoader(
    path="../1_Datasets/sample_pdf/",
    glob="*.pdf",
    loader_cls=PyPDFLoader # Need to mention the loader_cls here
)
dir_docs = dir_loader.load()
```

```
print("The data type of the document:", type(dir_docs))
print("The length of documents (number of total documents):",
len(dir_docs))
# print(dir_docs)
```

Some of the advantages to use DirectoryLoader:

- **Batch Processing**: Load multiple files at once
- **Customizable Patterns**: Use glob patterns to filter files
- **Loader Flexibility**: Specify any document loader class
- **Memory Efficiency**: Lazy loading option for large directories
- **Metadata Preservation**: Maintains source information